

Fast and Clean

AI-Assisted Research and Prototyping for New Clients

Reuben Brasher

Head of Customer Delivery Engineering, Terra Quantum AG

24 September 2026

The cold start

An opportunity in a domain you do not know, from people with no reason to trust you.

- The old arithmetic: weeks of unpaid work before anyone believes you.
- So you bluff, or you decline, or you over-invest in a stranger.
- **That arithmetic changed** — not because the model does the work, but because you can organise the work differently.

I do not mean a chat box

Not a website you paste a question into.

- An agent that runs **on your own machine**: reads your files, writes files, runs commands, reads what those commands printed.
- It can act, look at the result, and act again — with nobody in the loop between steps.
- Claude Code, Codex, and others. As a category, about two years old.
- If you have only used a chat interface, **this is a different thing**, and most of what you have read about prompting does not apply.

Why it changed how I think about work

The whole filesystem is available context.

- Your repositories, your notes, last week's downloads, a meeting transcript you forgot about.
- **It does not need to be told what is relevant.** Work that took days takes an afternoon. Understating that would be dishonest.
- It is also, word for word, the exposure.
- *Seeing everything you have* is one fact with two consequences. Everything else in this talk follows from that sentence.

Reasoning improved. The core business is still transformation.

Models reason better every year. That raises the ceiling on each step.

- It does not change what the machine is: a thing that turns one representation into another.
- My own slide, written months ago: “best understood as transformation, not reasoning or understanding.”
- Better reasoning makes each transformation bigger. **It does not remove the chain.**

So the job moved

- Not the reasoner. Not the typist.
- **The person who decides where the cuts go.**
- Three years ago the skill was prompting. Now it is decomposition.

Code used to be the asset. Now context is.

The inversion that makes all of this possible.

- Code was expensive and hard to replace. Documentation *described* it, and went stale.
- **That has inverted.** Specification, vocabulary and decision records now *determine* the code.
- Curated context, and an agent rewrites a codebase in hours. Code becomes a build artifact.
- Which is why developers started thinking like architects.

One engagement, five transformations

research → planning → data → code → hand-over

- Separate agent per stage. Their material does not cross. The last stage's *framing* does not cross. Context narrows.
- Roles follow for free: **rung** and **processor**.
- **Every boundary needs an artifact** — which is also the audit record.

Tuesday, week one. An email from Duddly's Duds.

A mid-market clothing chain with an unfortunate name and a real problem. They want better recommendations than they have. **Duds** from here on.

- You know recommender systems. You do not know fashion retail.
- Nothing is signed. Nobody has paid you.
- **The engagement has already begun.** From this email, anything their competitor should not see is already theirs.

The prompt I actually send

A recommender for a clothing retailer, strongly seasonal demand.

I already know: Hu/Koren/Volinsky, Koren, SASRec, two-tower.

I want: where calendar seasonality is first-class, not ordering or decay.

Surveys first, reputable venues, then fan out. Five areas, named.

Flag anything you are not confident exists.

- The seeds and the areas are my judgement. That is not delegated.

What it found, and what it nearly cost me

Clothing is seasonal. The research models time as *order* or *drift*. Neither says “week 47 looks like this for everybody.”

- **957 references checked.** The two literatures cite each other essentially zero times.
- **The perfect counterexample was withdrawn.** Unchecked, my strongest claim rests on a retracted paper.
- Same afternoon I scan their competitors. Reading list is **study**. The scan is **theirs**. *Same tool, same day, two owners.*

Thursday, week three. Two hours in a room.

Their sponsor, a brand manager, you, and one colleague. You are running an **event storm**: getting the domain out of their heads and into events, before anyone designs anything.

- No vocabulary agreed. No scope. No model of anything.
- This is where most projects are decided, and almost nothing is written down.

Do not ask it to design. Ask it to interview.

We are conducting an event storm on the topic of time-aware recommender systems in clothing retail.

- That is the entire prompt. A skill takes over and the agent becomes the facilitator: past-tense business events, no implementation, park what is uncertain.
- The tooling **refuses to invent**: a command with no trigger event is blocked.
- **The prompt got shorter because the method got packaged**. The work moved into a reusable definition.
- **The danger here is not error. It is fluency**. Asked to design, a model produces a coherent domain that is not yours.

The record keeps the disagreement

Every line is marked:

[owner] *is the owner's statement. Anything marked **[moderator]** is a question or an observation for the owner to confirm, and is not a fact.*

[owner] *One raw store per engagement [...] there is no global raw store. (The moderator first recorded this as a single global raw store; the owner corrected it.)*

- The assistant got the domain wrong and the correction survives.
- **That is the artifact — not the agreement, the disagreement.**

Monday, week four. The extract arrives.

- 31 million transactions. 1.4 million customers. 100,000 products. Two years.
- No documentation. Nobody at Duds can tell you how it was assembled.

“Profile it. Do not clean it.”

Illustrative, not verbatim — in practice several paragraphs carrying enough of the domain to be useful.

- Report what is wrong. Propose cleaning as a **list I approve**.
- The finding that matters is about **evaluation, not cleaning**: with seasonal data a random split leaks the future.
- **The danger here is tidiness**. A model that cleans unasked hands you a clean dataset and a wrong answer.

Their data, and the thing that moved

- Which model may see this is **decided per store and fails closed** — sensitive rows go to a local model, never into a session transcript, never to a hosted service.
- And data moves. Re-running a three-year-old experiment gave a result **10.6% different** — not a better model, a regenerated sample nobody recorded.
- **Identity is a digest, not a filename.**

Week six. Build something they can look at.

- This is the stage everyone thinks the talk is about.
- If the first three stages went well, it is the shortest part of the work.

Tests first, because wrong code does not crash

In data science a mistake returns **a number**, not an error.

- *Illustrative, not verbatim.* Characterization tests against current behaviour, then one change, its failing test, the command, the expected output.
- I write or approve the tests. The assistant does not grade its own work.
- **When the upstream stages went well this stage is boring, and boring is the whole objective.**
- Code declares where it came from exactly as documentation does. Both are deliverables.

Week ten. It works. Now you leave.

- The awkward part — and the part AI assistance makes materially worse if you are careless.
- *Illustrative, not verbatim.* Write the decision record now: the reasoning available at the time, including the assumption nobody tested and the thing you chose *not* to do.
- **Do not revise an executed decision to match what was built.** That fabricates foresight. Supersede it instead.

The deliverable is no longer where the value is

Your client contracts for a deliverable. They still do, and they are right to.

- The brief, the vocabulary, the decision records: **that is what would let anyone rebuild it.**
- The contract addresses the deliverable. It says nothing about the thing that now matters. **The industry has not caught up.**
- *And yes, this contradicts what I said two slides ago.* Nobody has drawn that line.

Whose directory is this?

A careless practitioner runs three engagements from one machine, one set of notes, one assistant.

- A brief conditioned on client A. A glossary derived from client B's vocabulary. A specification informed by both. Transcripts that touched all three.
- **Several clients could each make a legitimate claim on parts of that directory, and nobody could untangle which.**
- Not because anyone did anything wrong. Because the value moved somewhere the contracts do not describe.
- **This is what provenance is for.** Not to settle the question. To make sure you can still answer it when it is asked.

Five stages, five sessions, five documents

- Every other AI-risk talk warns that the model will leak your data.
- The likelier problem: you are **authoring** a record.
- Timestamped, ordered, and more revealing than any deliverable — because it shows how you thought, and beside what.

The ask is commercial, not legal

- Not a lawsuit. “Send us everything you used,” or a vendor questionnaire.
- **The rule has not been written.** I will not tell you what the law says. I will tell you what a client concludes.
- Not liability — the deal, and the referral behind it.
- **Would you be comfortable if Duds read your session history?**

One client per session, by construction

Not by remembering. The actual file my assistant reads before it does anything:

One project per session. *Directories are separate projects [...]. Stay in the one you were asked about.*

Do not open student-context, any engagement material, or llm-prompts.

- A rule that constrains the assistant, not the human.
- I have not opened those directories today, and the reason is four lines of a text file rather than my memory.
- Provenance does not hide the transcript. It makes it boring.

Every failure in this talk produced something plausible

- A believable empty search result. A tidy dataset. A metric. A branch name. A session history that looks like ordinary work.
- Yesterday I ran a documented command against a half-finished refactor. It did not fail. It wrote a confident wrong answer: 565 missing where there were none.
- **Nothing announces itself. That is what the chain is for.**

Fast and clean are one discipline

- Two months old, in use, increasingly tested, **not finished** — and a framework that looks finished and is not is worse than one that shows its gaps.
- If context is the asset, **contaminated context is not a compliance problem. It is the destruction of the only durable thing you own.**
- Tomorrow you pitch. What earns the work is not the demo. It is being someone who can be let near their data.